



A reinforcement learning based RMOEA/D for bi-objective fuzzy flexible job shop scheduling

Rui Li, Wenyin Gong*, Chao Lu*

School of Computer Science, China University of Geosciences, Wuhan 430074, China

ARTICLE INFO

Dataset link: <https://cuglirui.github.io/download.html>

Keywords:

Fuzzy flexible job shop scheduling
Multi-objective optimization
Parameter adaption
Reinforcement learning
MOEA/D

ABSTRACT

The flexible job shop scheduling problem (FJSP) is significant for realistic manufacturing. However, the job processing time usually is uncertain and changeable during manufacturing. This paper presents a multi-objective FJSP with fuzzy processing time (MOFFJSP) for optimizing the makespan and total machine workload as objectives. To solve the MOFFJSP, a MOEA/D based on reinforcement learning named RMOEA/D is proposed. RMOEA/D can be featured as: (i) an initial strategy with three rules is used to get a high-quality initial population; (ii) a parameter adaption strategy based on Q-learning is proposed to guide the population choose the best parameter to increase diversity; (iii) a variable neighborhood search based on reinforcement learning is designed to lead the solution to choose the right local search method; and (iv) an elite archive is used to improve the usage rate of the abandoned historical solution. RMOEA/D is compared with five well-known realted methods, i.e., MOEA/D, NSGA-II, MOEA/D-M2M, NSGA-III and IAIS on three benchmark suites. The results show that RMOEA/D outperforms these five state-of-art algorithms.

1. Introduction

1.1. Background

With the development of economic globalization, traditional flexible manufacturing met a quite huge challenge. And it is difficult to satisfy the requirement of the market (Lang et al., 2021; Rifai et al., 2021). Flexible job shop scheduling problem (FJSP) is a classical scheduling problem has been intensively studied over the past decades. Many heuristic algorithms have been proposed for FJSP such as genetic algorithm (GA) (Yuan et al., 2020), artificial bee colony algorithm (ABC) (Li, Huang, et al., 2020), two-phase meta-heuristic (Lei et al., 2019), Jaya (Caldeira & Gnanavelbabu, 2021), and teaching-learning-based optimization (TLBO) (Lei et al., 2018). However, fixed processing time is too idealized to simulate the practical manufacturing. In practical flexible manufacturing, the processing time is uncontrollable and it floats between an interval (Pan et al., 2021; Zhu & Zhou, 2021). So it is necessary to fuzzy the processing time for FJSP. FJSP with fuzzy processing time (FFJSP) is an extension of FJSP. FJSP has been proved as an NP-hard problem and FFJSP is also an NP-hard problem (Pavlov et al., 2019). Moreover, it is worth for studying how to efficiently solve FFJSP.

Furthermore, with the development of intelligent manufacturing and industrial 4.0, many industrial start to consider the low energy consumption manufacturing (Lu et al., 2021). Many energy-aware scheduling models have been proposed including distributed hybrid flow

shop (Shao et al., 2021b), distributed permutation flow shop (Lu et al., 2022), distributed flow shop with heterogeneous factory (Lu et al., 2021), distributed reentrant permutation flow shop scheduling (Rifai et al., 2021), distributed heterogeneous welding flow shop (Wang et al., 2021). However the energy aware FFJSP is seldom considered. The energy consumption is related to machine workload, idle time, and turn on/off times (Meng et al., 2020; Wang & Wang, 2021). And the main effect part of energy consumption is machine workload. Thus, multi-objective FFJSP (MOFFJSP) with minimizing makespan and total workload is significantly considered.

Inspired by scalar objective optimization problem, Zhang and Li (2007) proposed multi-objective evolutionary algorithm based on decomposition (MOEA/D) for multi-objective optimization problem (MoP). Based on reference vectors and the Tchebicheff function, MOEA/D can get good convergence and perform well diversity simultaneously. Recently, reinforcement learning (RL) is truing into a hot topic due to its strong ability of decision and optimization. Cooperation between the RL and evolutionary algorithms is a promising direction to solve complex optimization problems (Gong et al., 2021; Shiue et al., 2018; Zhao et al., 2019). Facing the complex optimization problems, it is expected to achieve superior performance via the synergy between RL and evolutionary computation as well as the problem-specific operators within the MOEA/D framework.

* Corresponding authors.

E-mail addresses: lirui@cug.edu.cn (R. Li), wygong@cug.edu.cn (W. Gong), luchao@cug.edu.cn (C. Lu).

1.2. Motivations

The MOFFJSP has been extensively studied for several years. However, in the existing literature, most methods execute local search strategies by poll pattern, which is inefficient and blind. Moreover, the performance of the algorithm is limited to parameter selection. A lot of black-box testing is performed to find the best parameter. Parameter selection problem lacks prior knowledge and it is time-consuming. To optimize makespan and total machine workload simultaneously, it is challenging and significant to develop effective algorithms for the MOFFJSP. Motivated by the above problems, this paper proposed an RL-based MOEA/D (RMOEA/D), which contains the following innovations: First in order to guide each solution adaptively to select the best local search strategy, a variable neighborhood search based on RL (RVNS) is proposed. Second, to make MOEA/D automatically adjust the parameter T , a parameter selection strategy based on Q-learning (Q-PAS) is designed. Next, an initial method integrating a variety of initial strategies is designed to get a high convergence and diverse population. Then, a discrete crossover and mutation method is used to obtain a large searching step. Moreover, an elite archive is applied to improve the utilization rate of abandoned solutions. Finally, to verify the performance of RMOEA/D, extensive numerical tests are carried out and the comparative results demonstrate the effectiveness of the above designs and the superiority of the proposed algorithm in solving the MOFFJSP.

1.3. Contributions

The main contributions of this paper go in five directions.

- (1) An initial strategy which combines the advantage of the three strategies is designed to provide an initial population with high convergence and diversity.
- (2) A parameter selection strategy based on Q-learning is proposed to let MOEA/D automatically select the best T to improve the diversity of PF.
- (3) A variable neighborhood search method based on RL is proposed to efficiently execute several local search strategies.
- (4) An elite archive is designed to collect the historical elite solution to increase the utilization rate of abandoned solutions.
- (5) The performance of RMOEA/D is executed on 23 FFJSP instances with different features. Experimental results show that RMOEA/D is superior to state-of-art algorithms under the condition of faster convergence.

The rest of this paper is organized as follows. Section 2 illustrates the literature review and some basic concept of triangular fuzzy processing time. In Section 3, the problem statement and modeling are introduced. In Section 4, the details of our approach RMOEA/D are reported. Numerical test experiments on RMOEA/D and discussion are shown in Section 5 and conclusions are summarized in Section 6.

2. Related work and background knowledge

In this section the literature of FFJSP, MOEA/D and RL applied to scheduling are briefly reviewed.

2.1. Fuzzy Flexible job shop scheduling problem

As an extension of FJSP, FFJSP has been researched for many years. Many excellent algorithms have been proposed for FFJSP including, GA (Lei, 2010, 2012), ABC (Gao, Suganthan, Pan, et al., 2016; Wang et al., 2013), discrete harmony search algorithm (Gao, Suganthan, Pan, & Tasgetiren, 2015), and TLBO (Lin, 2015; Xu et al., 2015). Recently, Sun et al. (2019) combine GA and PSO, which can efficiently solve different scales of problems. Lin (2019) design a backtracking

search strategy and utilize the hyper-level strategy to manipulate the low-level heuristics to obtain a good performance than other existing algorithms. Dorfeshan et al. (2020) abstract fuzzy processing time as a decision-maker and propose a weight distance approximate method, which can determine operation sequence better. Li, Liu, et al. (2020) propose a Type-2 fuzzy processing time to handle high uncertainty of complexity system, which supplements the disadvantage of the traditional triangular fuzzy number. Moreover, automatically adjusting population size is an efficient technique, but has not been applied to MOFFJSP. Pan et al. (2021) propose a bi-population evolutionary algorithm with feedback schema, which can make up for that gap. However, to the best of our knowledge, MOFFJSP has seldom been studied. So it is significant to design an algorithm for MOFFJSP.

2.2. MOEA/D

The MOEA/D is a classical multi-objective evolutionary algorithm (MOEA) that has been successfully applied for a variety of scheduling problems such as permutation flow shop scheduling problem (Pericleous et al., 2017), satellite range scheduling problem (Du et al., 2019), optimal power flow problem (Zhang et al., 2020), distributed heterogeneous hybrid flow shop scheduling problem (Shao et al., 2021a), and distributed heterogeneous welding flow shop scheduling problem (Wang et al., 2021). Referring to the superior performance of MOEA/D, the main motivations of using MOEA/D are following: (1) MOEA/D (Zhang & Li, 2007) has a great diversity to solve MoP. (2) RL can make MOEA/D select the best parameter and local search strategy which is more efficient than randomly selection. Thus, this paper applied an RL-based MOEA/D for MOFFJSP.

2.3. Reinforcement learning applied to scheduling

Recently, as a very popular artificial intelligence approach, RL has been widely applied in shop scheduling problems. Qu et al. (2015) developed a centralized RL approach for scheduling multi-state processes and multiple machines manufacturing systems. Qu et al. (2016) also designed a RL-based scheduling approach. It can adaptively update production plans by adopting real-time product and processing events information during executions. Wang and Yan (2016) proposed a knowledgeable multi-agents-based self-adaptive scheduling and adopted a dynamic scheduling strategy based on weighted Q-learning. To improve the performance, Shahrabi et al. (2017) put forward a Q-factor algorithm for parameter estimation to solve dynamic JSP with random job arrivals and machine breakdowns. Zhang et al. (2017) proposed simulation-based Q-learning for the scheduling problem from a Markov decision process perspective. Palombarini and Martínez (2018) adopted a deep RL approach that preserved the rescheduling knowledge in the deep Q-network in order to learn schedule repairing policies straightly from high-dimensional sensory inputs. Waschneck et al. (2018) applied a deep Q-network to semiconductor manufacturing scheduling and trained a deep neural network with flexible objectives. Shiue et al. (2018) designed a RL based approach including an off-line learning module and Q-learning. This method performs better than the previously dispatching rules. Ahmadi et al. (2018) modeled the job sequencing problem as a traveling salesman problem of second-order and solved it with a dynamic Q-learning-based genetic algorithm. Zhao et al. (2019) adopted a double-layer action Q-learning algorithm to solve dynamic FJSP and showed that this approach was suitable for dynamic FJSP. Palombarini and Martínez (2019) also applied the real-time rescheduling task as a closed-loop control problem and trained a deep Q-network to select repair actions in response to unexpected events and disturbances. Lin et al. (2019) proposed an edge computing-based smart manufacturing factory framework and used an improved deep Q-network to solve the JSP. Han and Yang (2020) designed a dueling double deep Q-network which combined the advantages of real-time response and flexibility of a deep convolutional neural

Table 1
Literature review of RL applied to scheduling.

References	Type of problem	Objective number	Machine number	Method
(Qu et al., 2015)	Dynamic job shop scheduling	S	M	Centralized RL
(Qu et al., 2016)	Dynamic job shop scheduling	S	M	RL
(Wang & Yan, 2016)	Knowledgeable manufacturing system	S	M	Weighted Q-learning
(Shahrabi et al., 2017)	Dynamic job shop scheduling	S	M	Q-factor algorithm
(Zhang et al., 2017)	Real-time job shop scheduling	S	M	Markov decision processes
(Palombarini & Martínez, 2018)	Socio-technical manufacturing system	S	M	Deep Q-learning
(Waschneck et al., 2018)	Job shop scheduling	S	M	Deep Q-network
(Shiue et al., 2018)	Real-time job shop scheduling	S	M	RL
(Ahmadi et al., 2018)	Job sequencing and tool switching problem	S	S	Q-learning
(Zhao et al., 2019)	Dynamic job shop scheduling	S	M	Q-learning
(Palombarini & Martínez, 2019)	Semiconductor production scheduling	S	M	Deep Q-network
(Lin et al., 2019)	Socio-technical manufacturing system	S	M	Deep Q-network
(Han & Yang, 2020)	Job shop scheduling	S	M	Dueling Double Deep Q-network
(Luo, 2020)	Dynamic job shop scheduling	S	M	Deep Q-learning
(Wang, 2020)	Dynamic job shop scheduling	M	M	Weighted Q-learning
(Liu et al., 2020)	Job shop scheduling	S	M	Actor-Critic DRL

network and RL and learned behavior strategies directly according to the input manufacturing states. Luo (2020) adopted deep Q-learning with a continuous state for dynamic FJSP, and it is superior to other dispatching rules. Wang (2020) proposed an adaptive strategy for FJSP with a weighted Q-learning algorithm by clustering to dynamically find the most suitable operation. Liu et al. (2020) viewed the JSP as a sequential decision-making problem and proposed deep RL to deal with it. To speed up model training, they also proposed a parallel training method that combined asynchronous updates with deep deterministic policy gradient. Table 1 showed literature summary about JSP with RL.

Although RL techniques are widely applied in manufacturing, it has not been considered in FFJSP yet. RL has a lot of advantages such as guide algorithm to select the best parameter or strategy and waste little time. So, RL is used for FFJSP to improve algorithm's performance.

2.4. Fuzzy Set

A fuzzy set $\tilde{F}$ contains two elements including x and membership function $\mu_{\tilde{F}}(x)$. $\mu_{\tilde{F}}(x)$ is the possibility of x belonging to $\tilde{F}$. All of x belong to a definite set X . The definition of fuzzy set is given as follows:

$$\tilde{F} = \{x, \mu_{\tilde{F}}(x) | \forall x \in X\}, 0 \leq \mu_{\tilde{F}}(x) \leq 1 \quad (1)$$

The classical set is definite. The fuzzy set transfer to classical set, when $\mu_{\tilde{F}}(x) = 1$,

$$F = \{x, \mu_{\tilde{F}}(x) = 1 | \forall x \in X\} \quad (2)$$

The most classical membership function in scheduling is triangle fuzzy number (TFN). As shown in Fig. 1, the membership function is similar to the triangle. t_1 is the earliest processing time, t_2 is the most possible processing time and t_3 is the latest processing time. A triple (t_1, t_2, t_3) is a TFN to represent the processing time. The formulation of the membership function is given as below:

$$\mu_{\tilde{F}}(x) = \begin{cases} 0, & x \leq t_1, \\ \frac{x - t_1}{t_2 - t_1}, & t_1 < x \leq t_2, \\ \frac{t_3 - x}{t_3 - t_2}, & t_2 < x < t_3, \\ 0, & x \geq t_3. \end{cases} \quad (3)$$

2.5. Fuzzy operators

Sakawa and Kubota (2000) define three operators in a fuzzy set, i.e., Addition operator, Ranking operator, and Max operator. For example, there are two TFNs $\tilde{s} = (s_1, s_2, s_3)$ and $\tilde{t} = (t_1, t_2, t_3)$.

(1) Addition operator: $\tilde{s} + \tilde{t} = (s_1 + t_1, s_2 + t_2, s_3 + t_3)$.

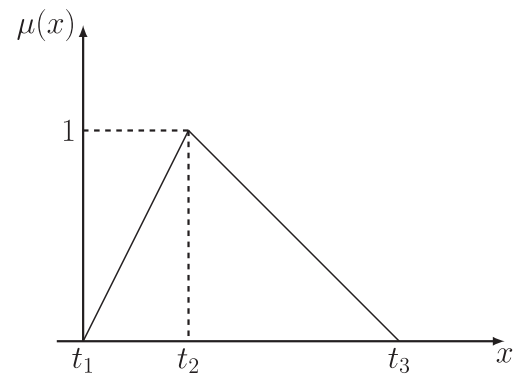


Fig. 1. Triangular membership function.

(2) Ranking operator: (a) $f_1(\tilde{x}) = \frac{x_1 + 2x_2 + x_3}{4}$. if $f_1(\tilde{s}) > f_1(\tilde{t})$, $\tilde{s} > \tilde{t}$, otherwise $\tilde{s} < \tilde{t}$. (b) $f_2(\tilde{x}) = x_2$, when $f_1(\tilde{s}) = f_1(\tilde{t})$, if $f_2(\tilde{s}) > f_2(\tilde{t})$, then $\tilde{s} > \tilde{t}$; otherwise $\tilde{s} < \tilde{t}$. (c) $f_3(\tilde{x}) = s_3 - s_1$, when $f_2(\tilde{s}) = f_2(\tilde{t})$, if $f_3(\tilde{s}) > f_3(\tilde{t})$, then $\tilde{s} > \tilde{t}$; otherwise $\tilde{s} < \tilde{t}$.

(3) Max operator: if $\tilde{s} > \tilde{t}$, then $\tilde{s} \vee \tilde{t} = \tilde{s}$; otherwise $\tilde{s} \vee \tilde{t} = \tilde{t}$.

3. Problem statement and mathematical modeling

3.1. Problem statement

A flexible job shop scheduling problem with fuzzy processing time from a real-world manufacturing process can be described as follow. $J = \{J_1, J_2, \dots, J_i, \dots, J_n\}$ is the job set and $\mathcal{M} = \{\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_k, \dots, \mathcal{M}_m\}$ is the machine set. Each job J_i has a set of Θ_i operations, $O_i = \{O_{i,1}, O_{i,2}, \dots, O_{i,j}, \dots, O_{i,\Theta_i}\}$, $O_{i,j} \in O_i$. Each operation can be processed on part of machines or all machines. And the processing time is a TFN $\tilde{P}_{i,j,k} = (p_1, p_2, p_3)$. FFJSP includes two subproblems, machine assignment and operation sequencing. The former is that each operation selects a machine from a candidate set. The latter is to schedule all operations on all machines to get feasible schedules. Some assumptions of FFJSP are given below:

- Each machine can process at most one operation at a time.
- No jobs may be processed on more than one machine at a time.
- The operation cannot be interrupted and the machine break down is not considered.
- Setup times and removal times are included in processing time
- Only if the previous operation of each job has been finished, the next operation can be handled.
- The processing time of each job is a TFN.

3.2. Notations

Before modeling this MOFFJSP, the notations used throughout the study are as follows:

- n : The number of jobs.
- m : The number of machines.
- i : The index of jobs.
- j : The index of operations.
- k : The index of machines.
- Θ_j : The total operation number of job J_i .
- $O_{i,j}$: The j_{th} operation of job J_i , $j = 1, \dots, \Theta_i$.
- $\tilde{S}_{i,j}$: The beginning time of operation $O_{i,j}$, which is a TFN: $\tilde{S}_{i,j} = (s_1, s_2, s_3)$.
- $\tilde{C}_{i,j}$: The finish time of operation $O_{i,j}$, which is a TFN: $\tilde{C}_{i,j} = (c_1, c_2, c_3)$.
- $\tilde{P}_{i,j,k}$: The processing time of j_{th} operation of job J_i on machine $\mathcal{M}_k$, which is a TFN: $\tilde{P}_{i,j,k} = (p_1, p_2, p_3)$
- $\mathbf{x}_{i,j,k}$: If the j_{th} operation of job J_i is processed on machine $\mathcal{M}_k$, the value is set to 1; otherwise is set to 0.
- $\mathbf{u}_{i_1,j_1,i_2,j_2}$: If the j_{2th} operation of job J_{i_2} is processed on machine $\mathcal{M}_k$ and machine $\mathcal{M}_k$ is also the candidate machine of the j_{1th} operation of job J_{i_1} , the value is set to 1; otherwise is set to 0.

3.3. The mathematical model of the MOFFJSP

The mathematical model of MOFFJSP is given as below:

$$\min F_1 = \max \left\{ \tilde{C}_{i,j_i} \right\}, 1 \leq i \leq n. \quad (4)$$

$$\min F_2 = \sum_{k=1}^m W_k. \quad (5)$$

$$W_k = \sum_{i=1}^n \sum_{j=1}^{\Theta_i} \tilde{P}_{i,j,k} * \mathbf{x}_{i,j,k}. \quad (6)$$

$$\tilde{C}_{i,j} = \tilde{S}_{i,j} + \tilde{P}_{i,j,k}. \quad (7)$$

$$\tilde{S}_{i,j} \geq \tilde{C}_{i-1,j}. \quad (8)$$

$$(\tilde{S}_{i_1,j_1} - \tilde{C}_{i_2,j_2}) * \mathbf{u}_{i_1,j_1,i_2,j_2} \geq 0. \quad (9)$$

$$\sum_{k=1}^m \mathbf{x}_{i,j,k} = 1. \quad (10)$$

$$\mathbf{x}_{i,j,k} = \begin{cases} 1, & \text{if } O_{ij} \text{ is processed on } \mathcal{M}_k, \\ 0, & \text{otherwise.} \end{cases} \quad (11)$$

$$\mathbf{u}_{i_1,j_1,i_2,j_2} = \begin{cases} 1, & \text{if } \mathbf{x}_{i_1,j_1,k} = \mathbf{x}_{i_2,j_2,k} = 1, \\ 0, & \text{otherwise.} \end{cases} \quad (12)$$

Eqs. (4) and (5) are two objective functions. Eq. (4) is the maximum fuzzy completion time of all operations and Eq. (5) is the total machine workload. Eq. (6) is a formulation to calculate machine $\mathcal{M}_k$'s workload. Eq. (7) describes the relationship between operation $O_{i,j}$'s completion time and starting time. Eq. (8) ensures operation $O_{i,j}$ must be processed after operation $O_{i,j-1}$ being finished. Eq. (9) specifies that the successor operation must wait for the following machine to be idle. Eq. (10) guarantees that job i has only one operation processed on the machine. The range of variables is described in Eqs. (11) and (12).

3.4. Illustrative example

To illustrate the problem, Fig. 2 presents a Gantt chart of a considered problem. This case has 3 jobs and 3 machines. All of the starting time and completion time is a TFN showing as a triangle in Fig. 2. However, if an operation is started at zero, it would be represented as a rectangle. Each transverse line simulates the operation flow on the machine. The TFN under the line is the fuzzy starting time and the above one is the fuzzy completion time. For example, the starting time of $O_{2,3}$ is (8,14,17) and the completion time is (17,24,28).

After computing the TFN of completion time for each operation, the maximum completion time and total machine workload for the system will be obtained, and the objective function can be collected.

4. Proposed algorithm

4.1. Framework of RMOEA/D

In this section, framework of the proposed algorithm MOEA/D based on RL is introduced, which is stated as following: first, the parameters and population are initialized. Then, perform variable neighborhood search for each solution to improve they exploitation. Next, apply Q-learning to select a parameter T for MOEA/D. Moreover, execute MOEA/D to generate new solution by using T . Finally, according to the change of PF's convergence and diversity, Q-table will be updated. And the elite archive will store the non-dominated solutions. If the stopping criterion is not satisfied go to row 4 and continue the iteration. Algorithm 1 describes the process of RMOEA/D.

Algorithm 1: RMOEA/D Algorithm.

Input: Population size N , mutation rate R , neighborhood number vector T , length of parameter memory LP , learning rate α , discount factor γ , greedy factor ϵ , elite archive $\mathbb{E}$

Output: the best solution found so far

- 1 Initialize the population $\mathbb{P}$ sizing N (c.f. Section 4.3)
- 2 Initialize all variables and parameters for algorithm.
- 3 **while** the stopping criterion is not satisfied **do**
- 4 Perform proposed VNS for each individual. (c.f. Section 4.6);
- 5 Apply Q-learning to allocate the parameter T for MOEA/D(c.f. Section 4.5);
- 6 Execute MOEA/D algorithm for each individual using parameter T_i by the discrete crossover and mutation methods. (c.f. Section 4.4);
- 7 Update population state and Q-table(c.f. Section 4.5);
- 8 Calculate the non-dominated solution set and collect them into elite archive(c.f. Section 4.7);

4.2. Encoding and decoding

In this paper, two one-dimensional vectors are used to represent the solution (or agent). The operation sequence is used to indicate the processing sequence for all operations. And the machine selection is used to represent the assigned machine for each operation. The two vectors are set with the same length which is equal to the total number of operations.

Fig. 3 displays an encoded solution. The solution representation contains two vectors. The operations sequence is $O_{3,1}, O_{2,1}, O_{1,1}, O_{2,2}, O_{1,2}, O_{3,2}, O_{1,3}, O_{3,3}, O_{2,3}$. The machine selection is $\mathcal{M}_1, \mathcal{M}_3, \mathcal{M}_2, \mathcal{M}_2, \mathcal{M}_2, \mathcal{M}_3, \mathcal{M}_2, \mathcal{M}_2, \mathcal{M}_1$. It is a one-to-one correspondence. For example, $O_{3,1}$ selected $\mathcal{M}_1$ and $O_{2,1}$ selected $\mathcal{M}_3$. The decoding of a solution is to assign the appropriate processing time for each operation on its selected machine according to the operation sequence. When a solution is decoded, the first vector in Fig. 3 is converted into a sequence of operations at first. Then each operation is assigned to a selected machine from the second vector in Fig. 3. Finally, the fuzzy processing times are assigned to the operation. In this paper, each solution (agent) is decoded into a fuzzy schedule. That is the processing time is a triangle fuzzy number.

4.3. Initialization strategy

In this section, an initial strategy integrated with 3 rules is introduced. Those rules includes Random rule (Gao, Suganthan, Pan, & Tasgetiren, 2015), local minimum processing time (LS) rule (Shaheed et al., 2018), global minimum workload (GW) rule (Li, Liu, et al., 2020). LS and GW are aiming at optimizing the objective function. The descriptions of three strategies are given as follows:

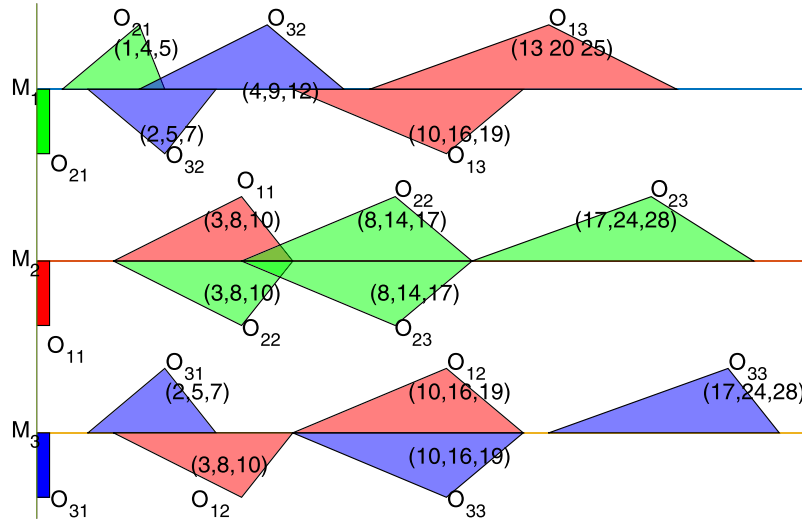


Fig. 2. A Gantt chart for an example problem.

Operation sequence	3	2	1	2	1	3	1	3	2
Machine selection	1	3	2	2	2	3	2	2	1

Fig. 3. Encoding representation.

Random: This rule is simple and ensures the initial population has high diversity. (1) repeat each job i for θ_i times to generate a scheduling vector. (2) randomly rearrange the sequence for all operations in the scheduling vector. (3) randomly choose a machine from the operation's candidate set for each operation and generate a routing vector.

LS: This rule aims to reduce the fuzzy makespan (maximum completion time). (1) randomly generate a scheduling vector same as the Random rule. (2) for each operation, choose the minimum processing time machine from the candidate set to generate the routing vector.

GW: This rule focuses on lower the total machine workload. (1) put $O_{1,1}, O_{2,1}, \dots, O_{N,1}$ into scheduling vector and rearrange the sequence. (2) rearrange the sequence of the rest operation and connect after the former sequence. (3) for each operation, choose an available machine with the minimum workload. If more than one machine had the same workload, the machine with minimum processing time would be selected.

The advantages of the three strategies are combined to obtain a population of high quality. A method called MIX3 is developed and the description is given in Algorithm 2.

Algorithm 2: MIX3 rule.

Input: Population size, Np .
Output: initialization population

- 1 Perform GW to generate an offspring $\mathbb{P}_1$, size $\lfloor Np/3 \rfloor$;
- 2 Execute LS to generate an offspring $\mathbb{P}_2$, size $\lfloor Np/3 \rfloor$;
- 3 Use Random to generate an offspring $\mathbb{P}_3$, size $\lfloor Np/3 \rfloor$;
- 4 Combine $\mathbb{P}_1$, $\mathbb{P}_2$ and $\mathbb{P}_3$ into *Parent* size $3 \times \lfloor Np/3 \rfloor$;
- 5 **if** size of *Parent* = Np **then**
- 6 terminate the algorithm;
- 7 **else**
- 8 supplement the rest of the solution with Random rule;

4.4. Crossover and mutation

To get a large exploring step, precedence operation crossover (POX) and universal crossover (UX) (Gao, Suganthan, Chua, et al., 2015) are adopted for MOFFJSP are applied. Fig. 4 gives two examples. The description is given as follows:

POX for operation sequence: (1) Randomly divide job set into two subset $\mathbb{J}_1$ and $\mathbb{J}_2$. (2) Select two solutions S_1 and S_2 . For each job belonging to $\mathbb{J}_1$, copy their operations into $NewS_1$. And for each job belonging to $\mathbb{J}_2$, copy their operations into $NewS_2$. (3) There exists too many space that is not filled with operation in $NewS_1$ and $NewS_2$. From the part of S_2 , copy the operation which does not appear in $NewS_1$ to the vacant positions in $NewS_1$ from left to right according to the order of the sequence in S_2 . And do the similar thing to $NewS_2$. The procedure is illustrated in Fig. 4.

UX for machine selection: (1) Randomly generate a 0-1 vector which length equals the total number of operations. (2) Select two solution M_1 and M_2 . Exchange the value in M_1 and M_2 , where it is 1 at the same position in 0-1 vector. The procedure is illustrated in Fig. 4.

Operation sequence mutation: randomly select two positions in the operation sequence and exchange the value. **Machine selection mutation:** randomly select two positions in the machine selection and choose a new machine from its candidate set.

4.5. Parameter adaption strategy based on Q-learning

This section will introduce the Q-learning algorithm and its detail design. Q-learning is a simple and useful RL technique because it can help algorithm select the best parameter during iteration, which can improve PF's diversity.

4.5.1. Brief introduction of Q-learning

Q-learning was first proposed in 1992 by Watkins and Dayan (1992), and it has been one of the most well-known RL algorithms. Q-learning consists of a quintuple $(\mathbb{A}, \mathbb{E}, C, S, R)$, which means Agents, Environment, Action set, State set and Reward. As shown in Fig. 5, the agent bases on its state S_t at time t in the environment and executes an action A_t . Then agent will get a reward R_{t+1} and its state will turn into a new state S_{t+1} . The Q value is updated based on the following formula:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha[R_t + \gamma \max(Q(S_{t+1}, A_t) - Q(S_t, A_t))] \quad (13)$$

Q-learning is a greedy algorithm and agent will select the action with biggest Q value to get the most reward. And by means of calculating

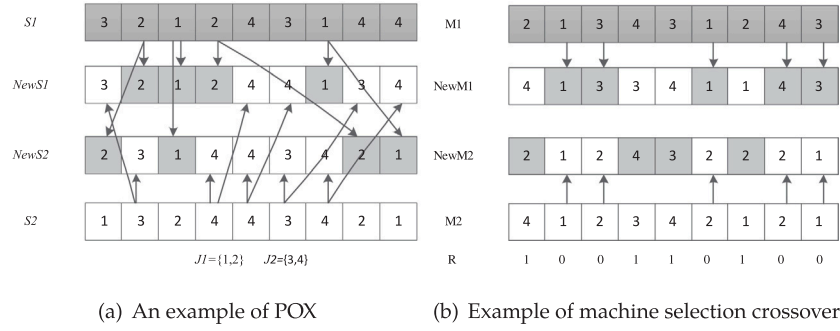


Fig. 4. An example of UX.

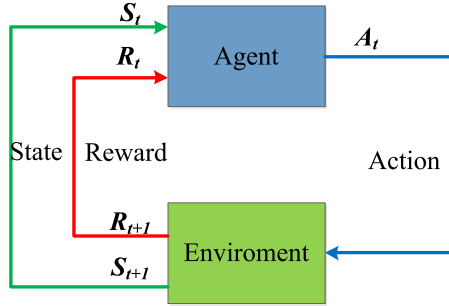


Fig. 5. The agent-environment interaction of RL.

the differential value between realistic Q value and estimated value, agent can adjust the gap between reality. α is the learning rate, which is between 0 and 1. γ is the discount factor is also in [0, 1]. When γ is close to 1, Q value is more affected by future state. On the contrary, the more γ is close to 0, Q value is more focus on the current state. It is a Markov decision process. The current state S_{t+1} can affect the later state S_{t+i} . R_t is the reward after executing action A_t .

4.5.2. Agent and action definition

In the MOEA, the Pareto Front (PF) is the best solution set calculated by MOEA and it reflects the algorithm's ability. By means of Q-learning, the population will be guided to choose the best parameter T to improve PF's diversity. Increasing the diversity of the whole population can improve the diversity of the PF. So the PF in each generation is abstracted as an agent to reflect the successful parameter selection. The four candidate values $T = 5, 10, 15, 20$ are defined as actions.

4.5.3. State definition

The change of agent's state will give Q-learning a feedback to judge whether the action executed this time can improve the PF's diversity or not. For describing the state, a convergence and diversity metric are defined. The equations are given as follows:

$$CV(P, P^*) = \frac{\sqrt{\sum_{y \in P} \min_{x \in P^*} dis(x, y)^2}}{|P|} \quad (14)$$

$$DV = \frac{\sum_{i=1}^{N-1} |d_i - \bar{d}|}{(N-1)\bar{d}} \quad (15)$$

$$\Delta CV = CV_{i-1} - CV_i \quad (16)$$

$$\Delta DV = DV_i - DV_{i-1} \quad (17)$$

CV (convergence) and DV (diversity) are referring to Deb et al. (2002) but have a little bit different. In Eq. (14), P is the PF calculated by the algorithm in each generation and P^* is the predefined reference

point because the true PF of the FFJSP instance cannot be calculated. The smaller CV is the better convergence is and $\Delta CV > 0$. In Eq. (15), d_i is the Euclid distance of two adjacent points in PF. $\bar{d}$ is the average value of d_i . The bigger DV is the better diversity is and $\Delta DV > 0$.

During the evolutionary, the ΔCV and ΔDV of PF have four combination conditions. (1) $\Delta CV > 0$ and $\Delta DV > 0$; (2) $\Delta CV > 0$ and $\Delta DV \leq 0$; (3) $\Delta CV \leq 0$ and $\Delta DV > 0$; (4) $\Delta CV \leq 0$ and $\Delta DV \leq 0$; (1) and (2) occurs during the period of evolutionary. (3) and (4) will happen when the population has converged. These four conditions are regarded as four states of agent.

4.5.4. Reward definition

After executing an action, the agent will get a reward, which might be positive or negative. The reward is defined as below:

$$Reward = \begin{cases} 10, & \Delta DV > 0, \\ 0, & \Delta DV \leq 0. \end{cases} \quad (18)$$

If the diversity of PF is better the chosen action (parameter) will get a reward and update the Q-table. On the contrary, the reward will be zero. It is worth noting that the reward value used most is 10 from the previous studies. We also used to consider a penalty when the agent fails to increase DV. However, this penalty will decrease the impact of successful reward. Thus, to let the positive reward more significant, the penalty is not considered in this work.

4.5.5. Q-learning for parameter self-adaption

Based on the illustration above, a parameter adaption strategy based on Q-learning namely Q-PAS is designed. Algorithm 3 described the details of Q-PAS. First, a Q-table is initiated to 0. Second, the agent's state S_t should be got. Next, ϵ -greedy selection strategy is applied to permit each action to have the probability to be selected. It is worth mentioning that the ϵ -greedy strategy is opposite to the traditional strategy, for the convenience of setting parameters in the experiment. Then, action A_t is executed to MOEA/D and PF P is got. Moreover, ΔCV and ΔDV are updated. The next state of agent S_{t+1} is got. Finally, $Q(S_t, A_t)$ in Q-table is updated by Eq. (15).

4.6. Variable neighborhood search based on RL

This section will introduce how to use RL to make solution adaptively select local search strategy. Local search is an essential technique to improve the exploitation. But it is time-consuming. When several local search methods are adopted, executing them one by one wastes too much time. If they were randomly executed, the successful rate would be low. However, RL can provide the guidance to help solution (agent) choose the local search strategy (action) most likely to succeed. In this paper five local search strategies are used. The descriptions are below:

LS1: Find the last finished operation O_{i,θ_i} , move O_{i,θ_i} to another machine $\mathcal{M}'$ with minimum processing time.

Algorithm 3: Parameter adaption strategy based on Q-learning (Q-PAS).

Input: Population $\mathbb{P}$, greedy factor ϵ , learning rate α , discount factor γ .

Output: Q_table

```

1 Q_table (4,4)  $\leftarrow$  0;  $\Delta CV \leftarrow 1$ ;  $\Delta DV \leftarrow 1$ ;
   $CV_{i-1} = CV_i = DV_{i-1} = DV_i = 0$ ;
2 while the stopping criterion is satisfied do
3    $CV_{i-1} = CV_i$ ;  $DV_{i-1} = DV_i$ ;
4   Calculate agent's state  $S_i$  (c.f. Section 4.5.3)
5   if rand  $< \epsilon$  then
6     Select the max  $Q(S_i, A_i)$  action  $A_i$ ,  $i = 1, 2, 3, 4$ 
7   else
8     Randomly select a action  $A_i$ 
9   Execute action  $A_i$  for MOEA/D to update  $\mathbb{P}$  and Get PF  $\mathcal{P}$ 
10  Calculate  $CV_i$  and  $DV_i$  by equation (16) and (17)
11   $\Delta CV = CV_{i-1} - CV_i$ ,  $\Delta DV = DV_i - DV_{i-1}$ 
12  Get action  $A_i$ 's reward  $R(S_i, A_i)$  by equation (20)
13  Calculate the new solution  $x_{new}$ 's state  $S_{i+1}$  (c.f. Section 4.5.3)
14   $Q(S_i, A_i) = Q(S_i, A_i) + \alpha[R(S_i, A_i) + \gamma \max(Q(S_{i+1}, A_i) - Q(S_i, A_i))]$ 

```

LS2: Randomly select an operation $O_{i,j}$ and move it to another machine with minimum processing time.

LS3: Find the maximum workload machine $\mathcal{M}$. Randomly select an operation which processed by $\mathcal{M}$ and move it to another machine $\mathcal{M}'$.

LS4: Randomly choose two positions on the operation sequence and exchange the value.

LS5: Randomly choose two positions on the operation sequence and insert the latter one in front of the former one.

There are two typical usages. (1) Randomly select one to execute from five methods; (2) Adopt all methods in turn. The first way has great blindness with low efficiency and the second way is time-consuming, especially when the solution has converged. So a variable neighborhood search based on RL namely RVNS is proposed to balance the advantages and disadvantages between the two methods mentioned above. Referring to SaDE (Qin et al., 2009), RVNS applies two memory matrices success memory SM and failure memory FM to record the success times and failure times of each local search method. By this means, RVNS can adjust the selected rate to make the strategy with a higher successful rate be selected more. Algorithm 4 gives the details. The description of RVNS is given as follows:

RVNS: (1) Execute roulette algorithm to assign a local search LS_i to each solution $\mathbb{P}(i)$. (2) Adopt local search LS_i to generate a new solution $\mathbb{P}'$ and judge whether the old solution can be updated. (3) Count the number of updating old solution successfully np and unsuccessfully nf as shown in Fig. 6. Save the record and insert it into the tail of success memory SM and failure memory FM . (4) If the length of SM and FM is longer than LP , delete the first record in SM and FM . Then update the probability of each parameter by sum each column in SM and FM as SR and FR and use SR to divide the sum of SR and FR . (5) Normalize the probability of each parameter $\mathcal{P}(i)$ to ensure the sum of probabilities equal to 1.

4.7. Elite archive

According to experimental visual observation, during the procedure of evolution, the good non-dominated solution in PF will be abandoned. Preserve those historical abandoned solutions can help the final PF have more elite solutions. So an elite archive is built to save the historical non-dominated solution. This archive's size is equal to population size. Algorithm 5 gives the archive update strategy. First archive $\mathcal{A}$ is combined with PF of this generation $\mathcal{PF}$. Then if the size is over Np , the fast non-dominated sort (Deb et al., 2002) is adopted to permit the size of $\mathcal{A}$ equal to Np .

Algorithm 4: Variable neighborhood search based on RL (RVNS).

Input: solution $\mathbb{P}(i)$, $\mathcal{P} = \{P_1, P_2, \dots, P_5\}$ selection probability of each local search, reference point Z^* , weight vector λ^i

Output: new solution $\mathbb{P}(i)'$

```

1 Execute Roulette Algorithm according to  $\mathcal{P}$  to assign a local search  $LS_i$  to  $\mathbb{P}(i)$ ;
2 Adopt  $LS_i$  to get a new solution  $\mathbb{P}(i)'$ ;
3 if  $g^{te}(\mathbb{P}(i)') | \lambda^i, Z^* < g^{te}(\mathbb{P}(i)) | \lambda^i, Z^*$  then
4    $\mathbb{P}(i) = \mathbb{P}(i)'$ ,  $ns_i = ns_i + 1$ 
5 else
6    $nf_i = nf_i + 1$ 
7 Count the  $ns_i$  and  $nf_i$   $i = 1, \dots, n$ . Save this record into the back of success memory and failure memory;
8 if length of  $SM > LP$  then
9   Delete the first record in  $SM$  and  $FM$ ;
10 for  $i = 1$  to  $n$  do
11    $SR(i) = \sum_{j=1}^{LP} ns_{i,j}$ ;  $FR(i) = \sum_{j=1}^{LP} nf_{i,j}$ ;
12    $\mathcal{P}(i) = \frac{SR(i)}{SR(i) + FR(i)}$ ;
13 for  $i = 1$  to  $n$  do
14    $\mathcal{P}(i) = \frac{\mathcal{P}(i)}{\sum_{i=1}^n \mathcal{P}(i)}$ 

```

Success Memory

Index	Parameter 1	Parameter 2	...	Parameter n
1	$ns_{1,G-LP}$	$ns_{2,G-LP}$	...	$ns_{n,G-LP}$
2	$ns_{1,G-LP+1}$	$ns_{2,G-LP+1}$	...	$ns_{n,G-LP+1}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
LP	$ns_{1,G-1}$	$ns_{2,G-1}$	...	$ns_{n,G-1}$

Failure Memory

Index	Parameter 1	Parameter 2	...	Parameter n
1	$nf_{1,G-LP}$	$nf_{2,G-LP}$	...	$nf_{n,G-LP}$
2	$nf_{1,G-LP+1}$	$nf_{2,G-LP+1}$	...	$nf_{n,G-LP+1}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
LP	$nf_{1,G-1}$	$nf_{2,G-1}$	...	$nf_{n,G-1}$

Fig. 6. Success memory and failure memory.

Algorithm 5: Update Elite Archive

Input: Archive $\mathcal{A}$, population size Np , PF of this generation $\mathcal{PF}$

Output: Archive $\mathcal{A}$

```

1  $\mathcal{A} \leftarrow \mathcal{A} \cup \mathcal{PF}$ ;
2 if length of  $\mathcal{A} > Np$  then
3    $\mathcal{A} \leftarrow \text{Non-dominated-sort}(\mathcal{A})$ 

```

4.8. Complexity of RMOEA/D

In this paper, RMOEA/D contains four components including MIX3, MOEA/D, Q-PAS, and RVNS. Their complexity is given as follows.

- On the MIX3, the complexity is $\mathcal{O}(Np)$, where Np is the size of the population.
- On the MOEA/D, the complexity is $\mathcal{O}(Iter * Np * 2 * T_i)$, where $Iter$ is the number of iteration and T_i is the number of the neighborhood.
- On the Q-PAS, the complexity is $\mathcal{O}(Iter)$, where LP is the length of success memory and failure memory and T is the vector of candidate parameter.

Table 2

Scale illustration of instances.

Name	N	M	SH	C-FFJSP	Literature
D1	10	10	40		
D2	10	10	40		
D3	10	10	50		
D4	10	10	50	Y	(Lei, 2010)
D5	15	10	80	Y	Lei (2012)
R1	5	4	23		
R2	8	8	64		
R3	10	6	81		
R4	10	10	100		
R5	15	8	171		
R6	15	10	185		
R7	20	10	308		
R8	20	15	355		(Gao, Suganthan, Pan, & Tasgetiren, 2015)
FMk1	10	6	55		
FMk2	10	6	58		
FMk3	15	8	150		
FMk4	15	8	90		
FMk5	15	4	106		
FMk6	10	15	150		
FMk7	20	5	100		
FMk8	20	10	225	N	
FMk9	20	10	240		(Brandimarte, 1993)
FMk10	20	15	240		

Table 3

Q-table after 200 iterations.

	$T = 5$	$T = 10$	$T = 15$	$T = 20$
State1 ($\Delta CV > 0, \Delta DV > 0$)	0	1.798693	0	6.743954
State2 ($\Delta CV > 0, \Delta DV \leq 0$)	0	0.323268	0	5.271637
State3 ($\Delta CV \leq 0, \Delta DV > 0$)	4.539988	0.766909	0	0
State4 ($\Delta CV \leq 0, \Delta DV \leq 0$)	2.847096	2.28377	2.911486	1.152641

Table 4Average rankings of the variant algorithms (Friedman) with p -value = 0.00015.

Algorithm	Ranking
RMOEA/D1	4.6087
RMOEA/D2	4.3913
RMOEA/D3	3.6087
RMOEA/D4	3.087
RMOEA/D5	2.913
RMOEA/D	2.3913

- On proposed the RVNS, the complexity is $\mathcal{O}(Iter * 5 * LP)$.
- On proposed the Elite archive, the complexity is $\mathcal{O}(Iter * Np^2)$.

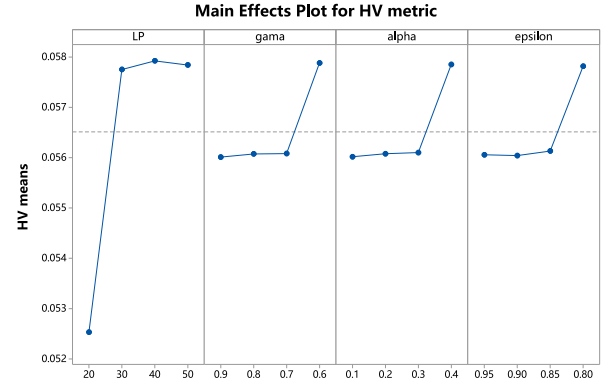
Therefore, the complexity of RMOEA/D is $\mathcal{O}(Iter * Np^2)$

5. Experimental results

In Section 4, the RMOEA/D algorithm has been described in details. In this section, we design detailed experiments to evaluate RMOEA/D's performance. RMOEA/D and the comparison algorithms are coded in MATLAB on an Intel Core i7 6700 CPU @ 3.4 GHz with 8G RAM. For fairness, all algorithm runs 30 independent times on each instance. Noting that, to verify the convergence and diversity of the proposed algorithm, after 30 independent runs, the average results are collected for performance comparison.

The comparison algorithms are well-known multi-objective optimization algorithms and their improved algorithms including MOEA/D (Zhang & Li, 2007), MOEA/D-M2M (Liu et al., 2014), NSGA-II (Deb et al., 2002), NSGA-III (Deb & Jain, 2014). A state-of-the-art algorithm improved artificial immune (IAIS) algorithm (Li, Liu, et al., 2020) is also compared.

Because the true PF (Deb et al., 2002) of manufacture cannot be calculated ahead. So we use Hypervolume (HV) (While et al., 2006)

**Fig. 7.** Main effects plot of HV metric.

value as the performance measures. HV is calculated as follows:

$$HV(P, r) = \bigcup_{x \in P} v(x, r). \quad (19)$$

where P is the PF calculated by algorithms, r is the reference point of all PFs, $r = (1, 1)$. x is the non-domination solution in PF and x needs to be normalized, which is $x = (\frac{f_1 - f_{1min}}{f_{1max} - f_{1min}}, \frac{f_2 - f_{2min}}{f_{2max} - f_{2min}})$. v is the volume of the hypercube which is bounded by x and r . HV unions all hypercubes for each non-domination solution x to generate an irregular hypercube and calculate its volume. It is worth mentioning that all x should be normalized in same reference point. The bigger HV is the convergence and diversity of algorithm is better.

5.1. Experimental instances

After introducing the experiment design, the experiment instances are reported. Three benchmarks are selected to verify the convergence and diversity of the proposed RMOEA/D. The first benchmark *Lei01* and *Lei02* are obtained from (Lei, 2010, 2012). The second benchmark *Remanu* is provided by (Gao, Suganthan, Pan, & Tasgetiren, 2015). All the instances are FFJSP with fuzzy processing time. Moreover, we transformed a flexible job shop scheduling problem benchmark *Mk* (Brandimarte, 1993). Refer to each processing time b , two integers a and c were randomly generated in interval $[0, b/2]$. (a, b, c) is a TFN. Therefore, *Mk* benchmark is converted to the FFJSP benchmark *FMk*. As shown in Table 2, the scale of all instances can be obtained.

5.2. Experimental parameters

After deciding the instances, the best performance parameters of the proposed algorithm can be found. The parameter configuration can impact the performance of the algorithm on solving this problem. The proposed RMOEA/D contains four parameters. The size of success and failure memory LP , the discount factor γ , the learning rate α and the greedy factor ϵ . A Taguchi approach of design-of-experiment (DOE) (Van Nostrand, 2002) is adopted. The parameter level is given as follows:

- $LP = 20, 30, 40, 50$.
- $\gamma = 0.9, 0.8, 0.7, 0.6$.
- $\alpha = 0.1, 0.2, 0.3, 0.4$.
- $\epsilon = 0.95, 0.90, 0.85, 0.80$.

An orthogonal array $L_{16}(4^4)$ is adopted in this calibration experiment. For fairness, each parameter runs 30 independent times. According to our previous work, the population size $Np = 100$ is the best. The other parameters include the max generation $G = 200$ and the neighborhood number vector T in Section 4.5 which has 4 candidate parameters $T = \{5, 10, 15, 20\}$. We collect the average HV value for

Table 5
Comparison of HV result for variants of RMOEA/D.

Instances	RMOEA/D1	RMOEA/D2	RMOEA/D3	RMOEA/D4	RMOEA/D5	RMOEA/D
D1	0.096358	0.097446	0.099396	0.098672	0.099119	0.099671
D2	0.101298	0.101899	0.103487	0.103056	0.103296	0.103148
D3	0.064942	0.067186	0.066809	0.067699	0.067898	0.06737
D4	0.057389	0.058733	0.058901	0.059791	0.058731	0.05847
D5	0.048071	0.050738	0.052962	0.053041	0.052376	0.052245
R1	0.050917	0.050711	0.050713	0.050458	0.050879	0.050919
R2	0.031251	0.032267	0.033204	0.031718	0.031889	0.031856
R3	0.034702	0.035149	0.035013	0.03601	0.036115	0.035549
R4	0.039239	0.039681	0.041548	0.041979	0.041942	0.042023
R5	0.042622	0.044959	0.045299	0.045912	0.046284	0.046105
R6	0.045522	0.050659	0.051547	0.052755	0.051897	0.051946
R7	0.041646	0.054828	0.056424	0.057074	0.057371	0.057465
R8	0.043417	0.069881	0.073654	0.073011	0.073377	0.074284
FMk01	0.058504	0.055246	0.056479	0.057164	0.056871	0.057207
FMk02	0.042559	0.040818	0.042238	0.043146	0.042341	0.042327
FMk03	0.06336	0.063915	0.064411	0.063918	0.064357	0.064147
FMk04	0.094665	0.095224	0.094696	0.095569	0.095808	0.09624
FMk05	0.048044	0.048418	0.048343	0.048325	0.048403	0.048468
FMk06	0.045506	0.043584	0.044403	0.045113	0.044132	0.044598
FMk07	0.071437	0.070497	0.070504	0.070188	0.070395	0.070645
FMk08	0.022032	0.021521	0.021134	0.021609	0.021323	0.021721
FMk09	0.042823	0.043149	0.042166	0.04264	0.042568	0.042508
FMk10	0.060198	0.062149	0.062444	0.062496	0.062798	0.063283

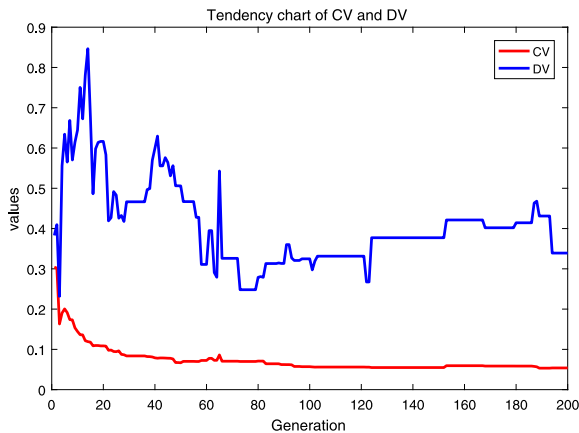


Fig. 8. Convergence and diversity change tendency graph.

30 runs. Fig. 7 shows the main effects plot of four parameters for HV metric. The higher the metric values is, the better the performance is. Based on the comprehensive observation, the best configuration of parametric value is set as $LP = 40$, $\gamma = 0.6$, $\alpha = 0.4$, and $\epsilon = 0.8$.

5.3. Discussion about Q-PAS

This section is to prove the effectiveness of Q-learning by picture and Q-table analysis. Fig. 8 shows the tendency chart of CV and DV of a one-time run. This example is a solution extract from instance D1. As shown in Fig. 8, each trough to peak can be regarded as a cycle. When population start to evolution, it converge fast because of local search, crossover, and mutation strategies. And DV reduces quickly. After exploring a period of time, solutions fall into local optima and Q-PAS guides population selects the best parameter to increase the diversity. So the DV increase fast. When next cycle starts, the regular for DV is obvious which reduces first and increase then. That can prove that Q-PAS has a great impact for increasing diversity when convergence is hard to improve. Table 3 gives the Q-table after 200 iterations. In the early period of iteration, CV reduce fast and T needs to be set bigger such as 20 to improve the DV. When solutions fall into local optima and CV is hard to increase, T need to be adjusted smaller such as 5 to increase DV. If both CV and DV seldom increase, then $T = 5$ or

10 may help the DV increase. From what has been discussed above, Q-PAS is an efficient strategy to let population dynamically select the best parameter in every state of agent.

5.4. Effectiveness of each improvement part of RMOEA/D

After proving the effectiveness of Q-learning the other improvement parts are verified to be efficiency in this section. We set several variants of RMOEA/D by adding the improvement part to MOEA/D step by step. RMOEA/D1 is the pure MOEA/D without any improvement, RMOEA/D2 is the MOEA/D with initial strategy, RMOEA/D3 is the RMOEA/D with randomly selection VNS, RMOEA/D4 is RMOEA/D3 with Q-PAS, RMOEA/D5 is RMOEA/D4 with Elite archive, RMOEA/D change VNS in RMOEA/D5 to RVNS. For fairness, all algorithms run 30 independent times. Table 5 shows the HV metric result of the comparison. Bold and gray value means the best result for each instance. And Friedman rank is shown in Table 4, each part improves the result against the last one. That proves the effectiveness of each part.

5.5. Comparison and discussion

5.5.1. Comparison with other algorithms

To further evaluate the performance of the proposed RMOEA/D algorithm. Five well-known algorithms are selected for comparison including MOEA/D, NSGA-II, MOEA/D-M2M, NSGA-III, and IAIS. For all compared algorithms, the related parameters are set referring to its original paper. And the results obtained after 30 independently runs are used to make detailed comparisons. The parameter is set as shown in Table 6.

Table 7 shows the HV results for algorithms comparison. The bold and gray value is the best average HV value in this instance. RMOEA/D is superior to other algorithms. And Table 8 shows the Friedman ranking of all algorithms. It can be concluded that RMOEA/D is the best algorithm on all instances and $P\text{-value} = 1.312 \times 10^{-17} < 0.05$. Table 9 shows the result of the Wilcoxon test. The results show that RMOEA/D is significantly better than other algorithms and all $p\text{-value}$ is smaller than 0.05.

Fig. 9 is the comparison of the PF. Three different scale problems are selected including D3, R5, and FMk10. Because of the improvement part of RMOEA/D. Its convergence and diversity are better than other algorithms. Fig. 10 shows the convergence comparison for all algorithms. The red line represents RMOEA/D and its convergence speed

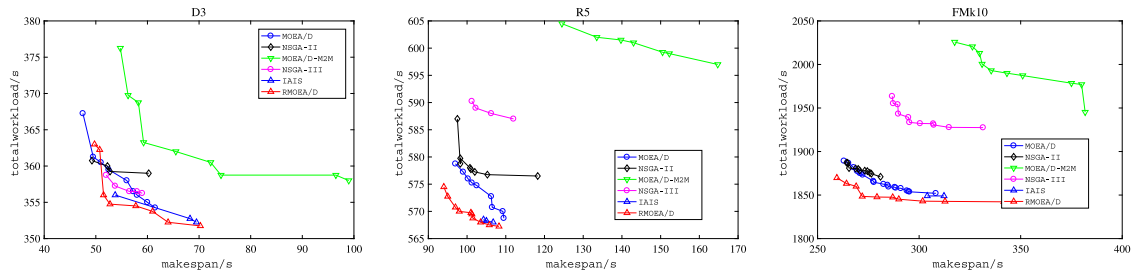


Fig. 9. Comparison results for Pareto Front.

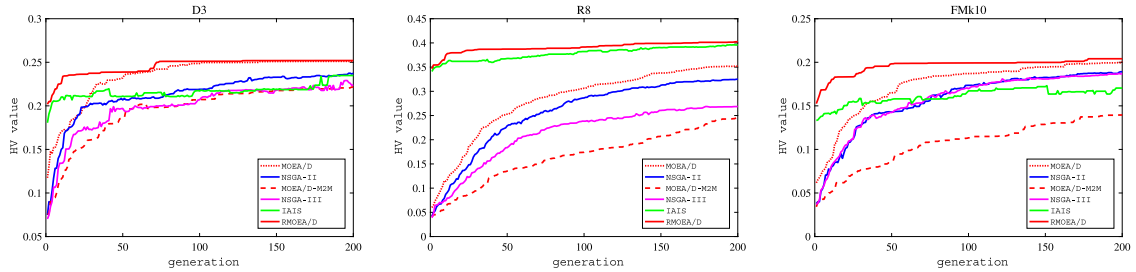


Fig. 10. Comparison results for metric convergence.

Table 6

Parameters setting.

Algorithm	Special parameters	Common parameters
RMOEA/D	memory size $LP = 40$, learning rate $\alpha = 0.4$, discount factor $\gamma = 0.6$, greedy factor $\epsilon = 0.8$, neighborhood vector $T = [5, 10, 15, 20]$	popsze $NP = 100$, weight vectors number $\lambda = 100$, mutation rate $R = 0.8$
MOEA/D	number of neighborhood $T = 10$	
MOEA/D-M2M	number of neighborhood $T = 10$, subpop size $S = 10$	
NSGA-III	number of neighborhood $T = 10$	
NSGA-II	–	$R = 0.8$, $Np = 100$
IAIS	clone-popsze $nc = 10$, popsze $NP = n_c * (n_c + 1)/2$, crowding degree threshold $CR_{max} = 1 * 10^{-4}$, temperature rate $w = 0.5$, initial temperature $T_0 = 1 * 10^4$	

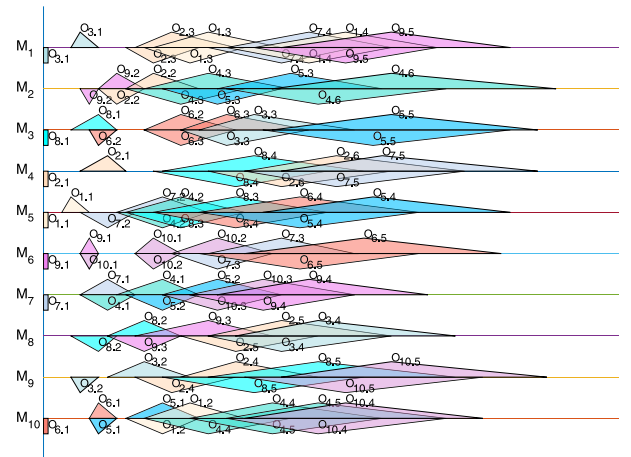


Fig. 11. The Gantt chart for solution with best makespan.

and final result are superior to others. In addition, the starting point RMOEA/D and IAIS are higher than others is because the proposed initial strategy generates an initial population with high convergence and diversity. Figs. 11 and 12 are two Gantt charts that represent the solution with the best makespan and another with the best total workload.

5.5.2. Discussion

The proposed RMOEA/D is better than other compared algorithms. There are four reasons below. (1) The initial strategy mentioned in Section 4.3 generated an initial population with high convergence and diversity, which results in the algorithm converging faster. (2) The parameter adaption strategy based on Q-learning proposed in Section 4.5 guides the algorithm chooses the most suitable parameter T to increasing the diversity of PF. (3) RMOEA/D adopted VNS based on RL described in Section 4.6 improves the convergence of the algorithm, so the results have better convergence. (4) The Elite archive collects

the abandoned solution with high quality during historical iteration. This supplemented the final PF and lead the RMOEA/D to have better results.

6. Conclusion

This paper proposed a RMOEA/D that combined two RL techniques to solve multi-objective fuzzy flexible job shop scheduling problems. The objective is to minimize the fuzzy makespan and total workload. As a classical adaption technique, RL can guide the algorithm to automatically select the best parameter or local search strategy. A novel Q-learning parameter adaption method including state definition, action definition, and reward definition is designed to help multi-objective optimization algorithm MOEA/D automatically select the parameter. Nevertheless, a RL method based on two historical memories is adopted to make the algorithm choose the local search strategy with the highest success probability. Next, to improve the usage rate of the historical solutions, an elite archive is used to collect the elite solution during iteration. Moreover, an initial strategy adopting three initial rules is applied to get an initial population with high convergence

Table 7
HV result for comparison with other algorithms.

Instances	IAIS	MOEA/D	MOEA/D-M2M	NSGA-II	NSGA-III	RMOEA/D
D1	0.101648	0.121501	0.102406	0.112091	0.10712	0.125066
D2	0.126441	0.145188	0.133167	0.135597	0.130158	0.147033
D3	0.107201	0.113763	0.096045	0.10405	0.099619	0.117255
D4	0.106326	0.111984	0.090837	0.103181	0.096756	0.113831
D5	0.14925	0.153384	0.11185	0.140018	0.129151	0.158382
R1	0.115783	0.123475	0.113377	0.121941	0.122881	0.123476
R2	0.067973	0.073794	0.061281	0.071102	0.068841	0.074325
R3	0.089022	0.093673	0.092318	0.091856	0.089554	0.09424
R4	0.096069	0.103368	0.082595	0.098912	0.093185	0.105898
R5	0.102766	0.107348	0.080571	0.098157	0.085406	0.111737
R6	0.135977	0.135997	0.07776	0.123481	0.105439	0.144087
R7	0.17597	0.170876	0.064412	0.150029	0.11291	0.19205
R8	0.193543	0.165819	0.068542	0.125642	0.083388	0.206107
FMk01	0.060938	0.075054	0.067977	0.070224	0.06702	0.073803
FMk02	0.058621	0.070369	0.062152	0.066106	0.064245	0.070362
FMk03	0.056997	0.078065	0.067212	0.07017	0.068431	0.07847
FMk04	0.068268	0.097126	0.082704	0.08333	0.077104	0.098794
FMk05	0.054107	0.06207	0.056213	0.056679	0.055799	0.062563
FMk06	0.096688	0.13492	0.098165	0.131115	0.116562	0.132173
FMk07	0.084151	0.099128	0.09029	0.091868	0.089918	0.099004
FMk08	0.038212	0.043297	0.036438	0.038692	0.038112	0.04306
FMk09	0.082438	0.092606	0.071231	0.089952	0.083816	0.09258
FMk10	0.153093	0.168807	0.123794	0.160561	0.148129	0.172348

Table 8
Average rankings of the comparison algorithms (Friedman) with p -value = 1.31285e-17.

Algorithm	Ranking
MOEA/D-M2M	5.2609
NSGA-III	4.6522
IAIS	4.6087
NSGA-II	3.3913
MOEA/D	1.8261
RMOEA/D	1.2609

Table 9
Results obtained by the Wilcoxon test for algorithm RMOEA/D.

VS	R^+	R^-	Exact p -value	Asymptotic p -value
IAIS	276.0	0.0	2.384E-7	0.000025
MOEA/D	237.0	39.0	0.0016598	0.002476
MOEA/D-M2M	276.0	0.0	2.384E-7	0.000025
NSGA-II	276.0	0.0	2.384E-7	0.000025
NSGA-III	276.0	0.0	2.384E-7	0.000025

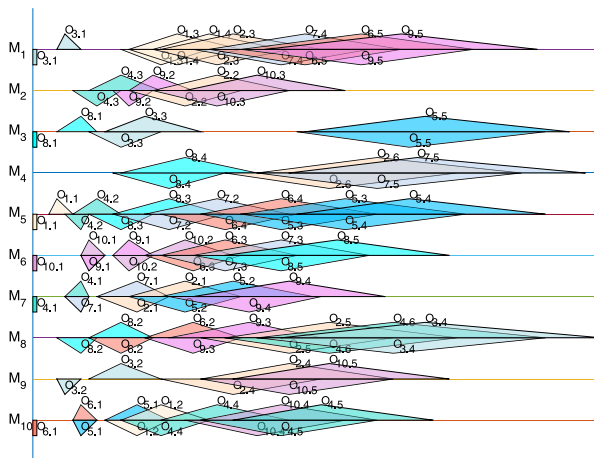


Fig. 12. The Gantt chart for solution with best total workload.

and diversity. Finally, the proposed RMOEA/D is compared with five well-known algorithms. Experimental results show that the proposed

algorithm is superior to others. In conclusion, the RMOEA/D is suitable to solve multi-objective fuzzy flexible job shop scheduling problems under a high level of uncertainty. And RL technique is an important method to realize parameter and strategy self-adaption.

Considering future research, first, it is interesting to increase the objective by more than three and applied RMOEA/D to solve the super multi-objective problem. Second, other types problem such as distribute flow job shop problem with fuzzy processing time is also of interest. Third, applied more complex techniques such as deep Q-network is another research direction.

CRedit authorship contribution statement

Rui Li: Resources, Project administration, Software, Data curation, Writing – original draft, Writing – review & editing. **Wenyin Gong:** Funding acquisition, Supervision, Conceptualization, Methodology, Writing – review & editing. **Chao Lu:** Methodology, Writing – review & editing.

Data availability

The dataset and code can be downloaded from <https://cuglirui.github.io/downloads.htm>

Acknowledgments

This work was partly supported by the National Natural Science Fund of China under Grant Nos. 62076225 and 62073300, and the Natural Science Fund for Distinguished Young Scholars of Hubei, China under Grant No. 2019CFA081. All authors approved the version of the manuscript to be published.

References

- Ahmadi, E., Goldengorin, B., Süer, G. A., & Mosadegh, H. (2018). A hybrid method of 2-TSP and novel learning-based GA for job sequencing and tool switching problem. *Applied Soft Computing*, 65, 214–229.
- Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41(3), 157–183.
- Caldeira, R. H., & Gnanavelbabu, A. (2021). A Pareto based discrete jaya algorithm for multi-objective flexible job shop scheduling problem. *Expert Systems with Applications*, 170, Article 114567.

- Deb, K., & Jain, H. (2014). An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints. *IEEE Transactions on Evolutionary Computation*, 18(4), 577–601.
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182–197.
- Dorfeshan, Y., Tavakkoli-Moghaddam, R., Mousavi, S. M., & Vahedi-Nouri, B. (2020). A new weighted distance-based approximation methodology for flow shop scheduling group decisions under the interval-valued fuzzy processing time. *Applied Soft Computing*, 91, Article 106248.
- Du, Y., Xing, L., Zhang, J., Chen, Y., & He, Y. (2019). MOEA based memetic algorithms for multi-objective satellite range scheduling problem. *Swarm and Evolutionary Computation*, 50, Article 100576.
- Gao, K. Z., Suganthan, P. N., Chua, T. J., Chong, C. S., Cai, T. X., & Pan, Q. K. (2015). A two-stage artificial bee colony algorithm scheduling flexible job-shop scheduling problem with new job insertion. *Expert Systems with Applications*, 42(21), 7652–7663.
- Gao, K. Z., Suganthan, P. N., Pan, Q. K., Chua, T. J., Chong, C. S., & Cai, T. X. (2016). An improved artificial bee colony algorithm for flexible job-shop scheduling problem with fuzzy processing time. *Expert Systems with Applications*, 65, 52–67.
- Gao, K. Z., Suganthan, P. N., Pan, Q. K., & Tasgetiren, M. F. (2015). An effective discrete harmony search algorithm for flexible job shop scheduling problem with fuzzy processing time. *International Journal of Productions Research*, 53(19), 5896–5911.
- Gong, W., Liao, Z., Mi, X., Wang, L., & Guo, Y. (2021). Nonlinear equations solving with intelligent optimization algorithms: A survey. *Complex System Modeling and Simulation*, 1(1), 15–32.
- Han, B. A., & Yang, J. J. (2020). Research on adaptive job shop scheduling problems based on dueling double DQN. *IEEE Access*, 8, 186474–186495.
- Lang, S., Reggelin, T., Schmidt, J., Müller, M., & Nahhas, A. (2021). NeuroEvolution of augmenting topologies for solving a two-stage hybrid flow shop scheduling problem: A comparison of different solution strategies. *Expert Systems with Applications*, 172, Article 114666.
- Lei, D. (2010). A genetic algorithm for flexible job shop scheduling with fuzzy processing time. *International Journal of Productions Research*, 48(10), 2995–3013.
- Lei, D. (2012). Co-evolutionary genetic algorithm for fuzzy flexible job shop scheduling. *Applied Soft Computing*, 12(8), 2237–2245.
- Lei, D., Gao, L., & Zheng, Y. (2018). A novel teaching-learning-based optimization algorithm for energy-efficient scheduling in hybrid flow shop. *IEEE Transactions on Engineering Management*, 65(2), 330–340.
- Lei, D., Li, M., & Wang, L. (2019). A two-phase meta-heuristic for multiobjective flexible job shop scheduling problem with total energy consumption threshold. *IEEE Transactions on Cybernetics*, 49(3), 1097–1109.
- Li, Y., Huang, W., Wu, R., & Guo, K. (2020). An improved artificial bee colony algorithm for solving multi-objective low-carbon flexible job shop scheduling problem. *Applied Soft Computing*, 95, Article 106544.
- Li, J., Liu, Z., Li, C., & Zheng, Z. (2020). Improved artificial immune system algorithm for type-2 fuzzy flexible job shop scheduling problem. *IEEE Transactions on Fuzzy Systems*, 1.
- Lin, J. (2015). A hybrid biogeography-based optimization for the fuzzy flexible job-shop scheduling problem. *Knowledge-Based Systems*, 78, 59–74.
- Lin, J. (2019). Backtracking search based hyper-heuristic for the flexible job-shop scheduling problem with fuzzy processing time. *Engineering Applications of Artificial Intelligence*, 77, 186–196.
- Lin, C., Deng, D., Chih, Y., & Chiu, H. (2019). Smart manufacturing scheduling with edge computing using multiclass deep Q network. *IEEE Transactions on Industrial Informatics*, 15(7), 4276–4284.
- Liu, C., Chang, C., & Tseng, C. (2020). Actor-critic deep reinforcement learning for solving job shop scheduling problems. *IEEE Access*, 8, 71752–71762.
- Liu, H., Gu, F., & Zhang, Q. (2014). Decomposition of a multiobjective optimization problem into a number of simple multiobjective subproblems. *IEEE Transactions on Evolutionary Computation*, 18(3), 450–455.
- Lu, C., Gao, L., Yi, J., & Li, X. (2021). Energy-efficient scheduling of distributed flow shop with heterogeneous factories: A real-world case from automobile industry in China. *IEEE Transactions on Industrial Informatics*, 17(10), 6687–6696.
- Lu, C., Huang, Y., Meng, L., Gao, L., Zhang, B., & Zhou, J. (2022). A Pareto-based collaborative multi-objective optimization algorithm for energy-efficient scheduling of distributed permutation flow-shop with limited buffers. *Robotics and Computer-Integrated Manufacturing*, 74, Article 102277.
- Luo, S. (2020). Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Applied Soft Computing*, 91, Article 106208.
- Meng, L., Ren, Y., Zhang, B., Li, J.-Q., Sang, H., & Zhang, C. (2020). MILP modeling and optimization of energy-efficient distributed flexible job shop scheduling problem. *IEEE Access*, 8, 191191–191203.
- Palombarini, J. A., & Martínez, E. C. (2018). Automatic generation of rescheduling knowledge in socio-technical manufacturing systems using deep reinforcement learning. In *2018 IEEE biennial congress of argentina* (pp. 1–8).
- Palombarini, J. A., & Martínez, E. C. (2019). Closed-loop rescheduling using deep reinforcement learning. *IFAC-PapersOnLine*, 52(1), 231–236.
- Pan, Z., Lei, D., & Wang, L. (2021). A bi-population evolutionary algorithm with feedback for energy-efficient fuzzy flexible job shop scheduling. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 1–13.
- Pavlov, A. A., Misura, E. B., Melnikov, O. V., & Mukha, I. P. (2019). NP-hard scheduling problems in planning process automation in discrete systems of certain classes. In Z. Hu, S. Petoukhov, I. Dychka, & M. He (Eds.), *Advances in computer science for engineering and education* (pp. 429–436). Cham: Springer International Publishing.
- Pericleous, S., Konstantinidis, A., Achilleos, A., & Papadopoulos, G. A. (2017). Generic hybridization of MOEA/D with learning for permutation flow shop scheduling problem. In *2017 8th International conference on information, intelligence, systems & applications* (pp. 1–6).
- Qin, A. K., Huang, V. L., & Suganthan, P. N. (2009). Differential evolution algorithm with strategy adaptation for global numerical optimization. *IEEE Transactions on Evolutionary Computation*, 13(2), 398–417.
- Qu, S., Chu, T., Wang, J., Leckie, J., & Jian, W. (2015). A centralized reinforcement learning approach for proactive scheduling in manufacturing. In *2015 IEEE 20th conference on emerging technologies and factory automation* (pp. 1–8).
- Qu, S., Wang, J., & Shivani, G. (2016). Learning adaptive dispatching rules for a manufacturing process system by using reinforcement learning approach. In *2016 IEEE 21st international conference on emerging technologies and factory automation* (pp. 1–8).
- Rifai, A. P., Mara, S. T. W., & Sudiarso, A. (2021). Multi-objective distributed reentrant permutation flow shop scheduling with sequence-dependent setup time. *Expert Systems with Applications*, 183, Article 115339.
- Sakawa, M., & Kubota, R. (2000). Fuzzy programming for multiobjective job shop scheduling with fuzzy processing time and fuzzy due date through genetic algorithms. *European Journal of Operational Research*, 120(2), 393–407.
- Shaheed, I. M., Shukor, S. A., & Abdullah, S. (2018). Population initialisation methods for fuzzy job-shop scheduling problems: Issues and future trends. *International Journal on Advanced Science, Engineering and Information Technology*, 8(4–2), 1820–1828.
- Shahrabi, J., Adibi, M. A., & Mahootchi, M. (2017). A reinforcement learning approach to parameter estimation in dynamic job shop scheduling. *Computers & Industrial Engineering*, 110, 75–82.
- Shao, W., Shao, Z., & Pi, D. (2021a). An ant colony optimization behavior-based MOEA/D for distributed heterogeneous hybrid flow shop scheduling problem under nonidential time-of-use electricity tariffs. *IEEE Transactions on Automation Science and Engineering*, 1–16.
- Shao, W., Shao, Z., & Pi, D. (2021b). Multi-objective evolutionary algorithm based on multiple neighborhoods local search for multi-objective distributed hybrid flow shop scheduling problem. *Expert Systems with Applications*, 183, Article 115453.
- Shiue, Y.-R., Lee, K.-C., & Su, C.-T. (2018). Real-time scheduling for a smart factory using a reinforcement learning approach. *Computers & Industrial Engineering*, 125, 604–614.
- Sun, L., Lin, L., Gen, M., & Li, H. (2019). A hybrid cooperative coevolution algorithm for fuzzy flexible job shop scheduling. *IEEE Transactions on Fuzzy Systems*, 27(5), 1008–1022.
- Van Nostrand, R. C. (2002). Design of experiments using the taguchi approach: 16 steps to product and process improvement. *Technometrics*, 44(3), 289.
- Wang, Y.-F. (2020). Adaptive job shop scheduling strategy based on weighted Q-learning algorithm. *Journal of Intelligent Manufacturing*, 31(2), 417–432.
- Wang, G., Li, X., Gao, L., & Li, P. (2021). Energy-efficient distributed heterogeneous welding flow shop scheduling problem using a modified MOEA/D. *Swarm and Evolutionary Computation*, 62, Article 100858.
- Wang, J.-j., & Wang, L. (2021). A cooperative memetic algorithm with learning-based agent for energy-aware distributed hybrid flow-shop scheduling. *IEEE Transactions on Evolutionary Computation*, 1. <http://dx.doi.org/10.1109/TEVC.2021.3106168>.
- Wang, H.-X., & Yan, H.-S. (2016). An interoperable adaptive scheduling strategy for knowledgeable manufacturing based on SMGWQ-learning. *Journal of Intelligent Manufacturing*, 27(5), 1085–1095.
- Wang, L., Zhou, G., Xu, Y., & Liu, M. (2013). A hybrid artificial bee colony algorithm for the fuzzy flexible job-shop scheduling problem. *International Journal of Productions Research*, 51(12), 3593–3608.
- Waschneck, B., Reichstaller, A., Belzner, L., Altenmüller, T., Bauernhansl, T., Knapp, A., & Kyek, A. (2018). Deep reinforcement learning for semiconductor production scheduling. In *2018 29th Annual SEMI advanced semiconductor manufacturing conference* (pp. 301–306).
- Watkins, C. J. C. H., & Dayan, P. (1992). Technical note: Q-learning. *Machine Learning*, 8(3), 279–292.
- While, L., Hingston, P., Barone, L., & Huband, S. (2006). A faster algorithm for calculating hypervolume. *IEEE Transactions on Evolutionary Computation*, 10(1), 29–38.
- Xu, Y., Wang, L., Wang, S.-y., & Liu, M. (2015). An effective teaching-learning-based optimization algorithm for the flexible job-shop scheduling problem with fuzzy processing time. *Neurocomputing*, 148, 260–268.

- Yuan, S., Li, T., & Wang, B. (2020). A co-evolutionary genetic algorithm for the two-machine flow shop group scheduling problem with job-related blocking and transportation times. *Expert Systems with Applications*, 152, Article 113360.
- Zhang, Q., & Li, H. (2007). MOEA/D: a multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6), 712–731.
- Zhang, T., Xie, S., & Rose, O. (2017). Real-time job shop scheduling based on simulation and Markov decision processes. In *WSC '17, Proceedings of the 2017 winter simulation conference*. IEEE Press.
- Zhang, J., Zhu, X., & Li, P. (2020). MOEA/D with many-stage dynamical resource allocation strategy to solution of many-objective OPF problems. *International Journal of Electrical Power & Energy Systems*, 120, Article 106050.
- Zhao, M., Li, X., Gao, L., Wang, L., & Xiao, M. (2019). An improved Q-learning based rescheduling method for flexible job-shops with machine failures. In *2019 IEEE 15th international conference on automation science and engineering* (pp. 331–337).
- Zhu, Z., & Zhou, X. (2021). A multi-objective multi-micro-swarm leadership hierarchy-based optimizer for uncertain flexible job shop scheduling problem with job precedence constraints. *Expert Systems with Applications*, 182, Article 115214.